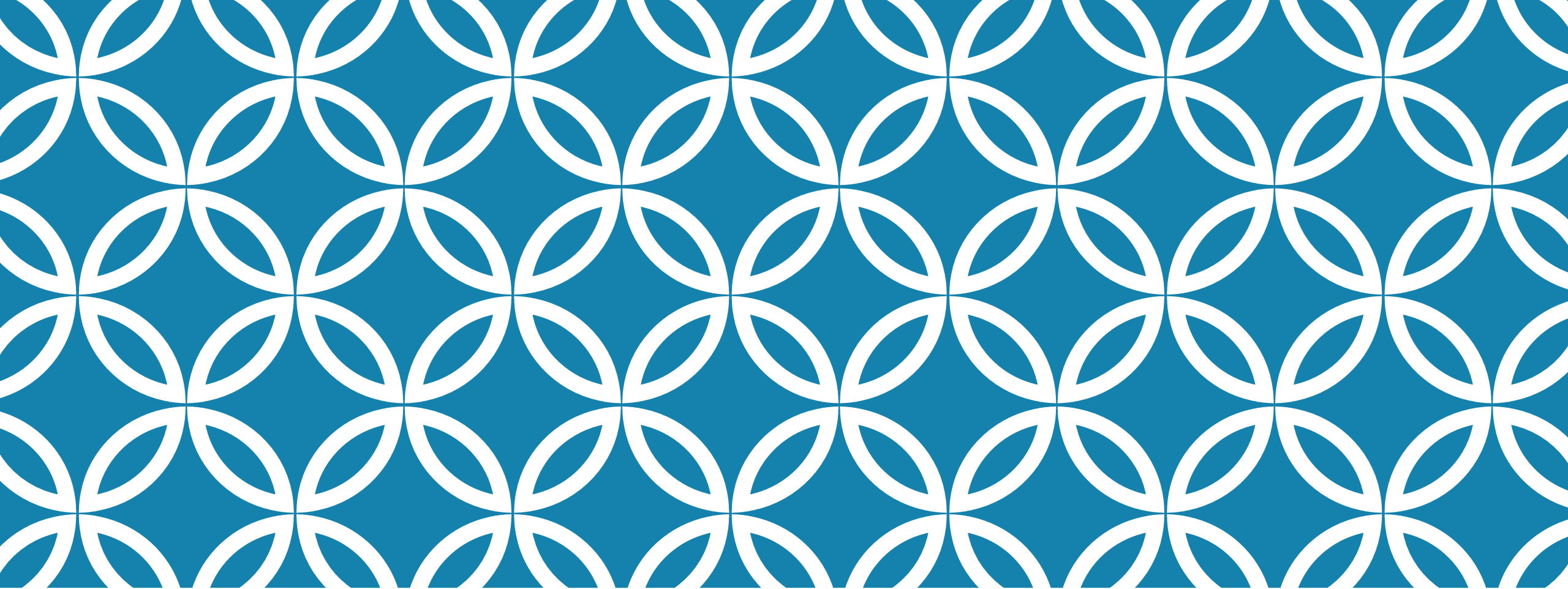




DEPLOYING HIGHLY AVAILABLE WEB APPLICATION ON AWS WITH TERRAFORM

HUG BLR meetup#01
2nd November 2025
Couchbase office , Bengaluru

AGENDA

An application simple Architecture

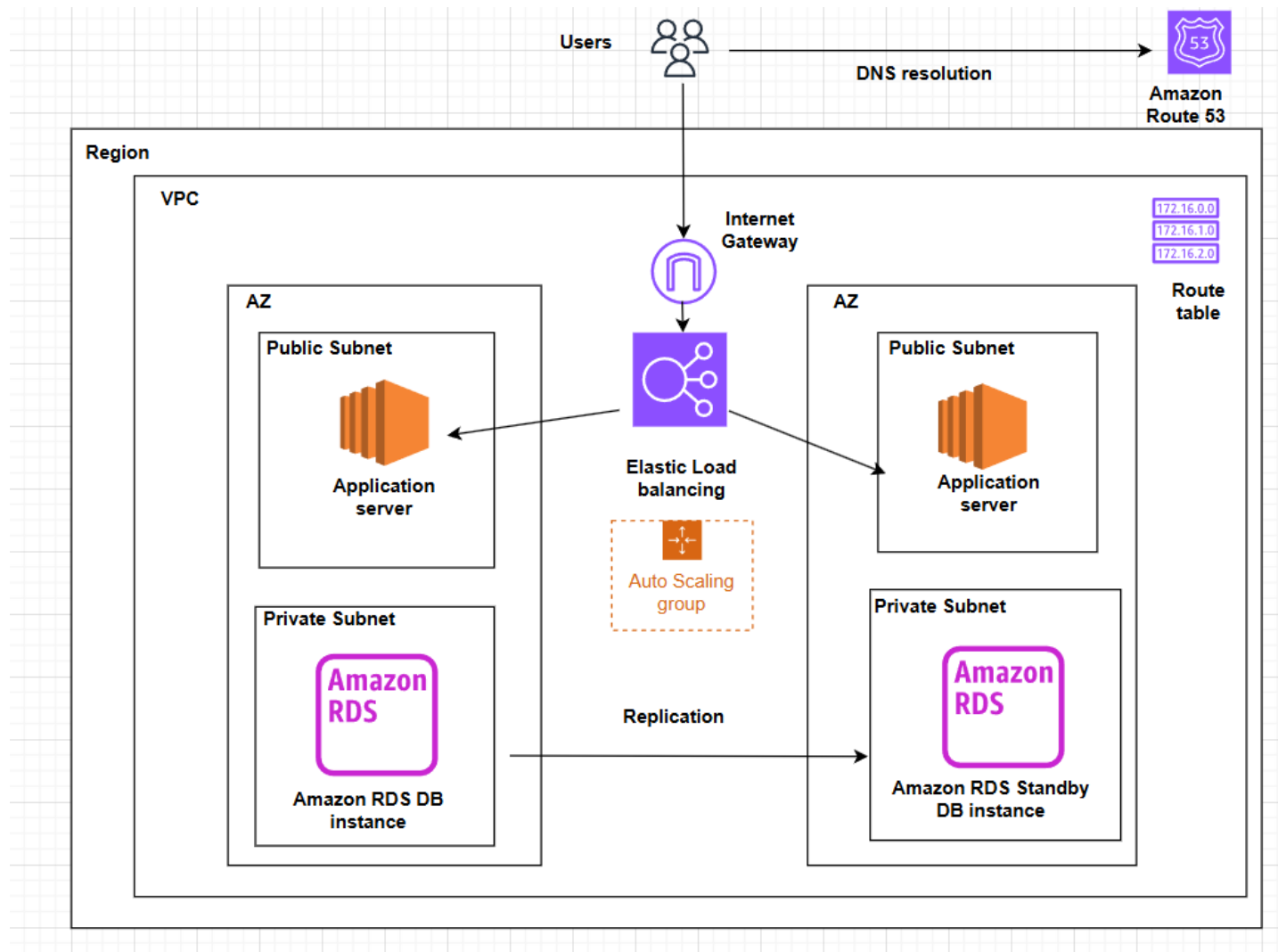
AWS services and components

- EC2
- VPC
- Application load balancer
- Security Groups

Terraform introduction

Demo

AN APPLICATION SIMPLE ARCHITECTURE



INFRASTRUCTURE AS CODE WITH TERRAFORM

Infrastructure as Code (IaC) tools allows to **manage infrastructure with configuration files**

IaC allows to build, change, and manage infrastructure in a safe, consistent, and repeatable way by defining resource configurations that can version, reuse, and share.

Terraform is **HashiCorp's** infrastructure as code tool.

It lets define resources and infrastructure in

- human-readable,
- declarative configuration files, and
- manages infrastructure's lifecycle.

INFRASTRUCTURE AS CODE WITH TERRAFORM CONT..

Using Terraform has several advantages over manually managing infrastructure:

1. Terraform can manage infrastructure on multiple cloud platforms.
2. The human-readable configuration language helps to write infrastructure code quickly.
3. **Terraform's state** allows to track resource changes throughout deployments.

STANDARDIZE DEPLOYMENT WORKFLOW

Terraform plugins called **providers** let Terraform interact with cloud platforms and other services via their application programming interfaces (APIs).

Providers define individual units of infrastructure, for example compute instances or private networks, as resources.

Terraform's configuration language is **declarative**, it describes the desired end-state for infrastructure

Terraform providers **automatically calculate dependencies between resources** to create or destroy them in the correct order.

TO DEPLOY INFRASTRUCTURE WITH TERRAFORM:

Scope - Identify the infrastructure for a project.

Author - Write the configuration for your infrastructure.

Initialize - Install the plugins Terraform needs to manage the infrastructure.

Plan - Preview the changes Terraform will make to match your configuration.

Apply - Make the planned changes.

TRACK INFRASTRUCTURE

Terraform keeps track of real infrastructure in a **state file**, which acts as a source of truth for environment.

Terraform uses the state file to determine the changes to make to the infrastructure so that it will match configuration.

INSTALL TERRAFORM

<https://www.terraform.io/>

BUILD INFRASTRUCTURE

Prerequisites

The Terraform CLI (1.2.0+) installed.

The AWS CLI installed.

AWS account and associated credentials that allow you to create resources.

To use your IAM credentials to authenticate the Terraform AWS provider, set the `AWS_ACCESS_KEY_ID` environment variable.

```
$ export AWS_ACCESS_KEY_ID=
```

Now, set your secret key.

```
$ export AWS_SECRET_ACCESS_KEY=
```

WRITE CONFIGURATION

The set of files used to describe infrastructure in Terraform is called as a **Terraform *configuration***.

The ***terraform* {}** block contains Terraform settings, including the required providers Terraform will use to provision infrastructure.

The provider block configures the specified provider.

A provider is a **plugin** that Terraform uses to create and manage resources.

RESOURCES

- Use **resource blocks** to define components of infrastructure.
- Resource blocks have two strings before the block:
 - the resource type and the resource name.
- In this example, the resource type is `aws_instance` and the name is `app_server`.
- The prefix of the type maps to the name of the provider. In the example configuration, Terraform manages the `aws_instance` resource with the `aws` provider.
- Together, the resource type and resource name form a unique ID for the resource.
- For example, the ID for your EC2 instance is `aws_instance.app_server`.

RESOURCES CONT..

- Resource blocks contain arguments to configure the resource.
- Arguments can include things like machine sizes, disk image names, or VPC IDs
- For EC2 instance, the example configuration sets the AMI ID to an Ubuntu image, and the instance type to t2.micro,

INITIALIZE THE DIRECTORY

- When you create a new configuration, you need to initialize the directory with **terraform init**.
- Initializing a configuration directory downloads and installs the providers defined in the configuration
- Terraform downloads the provider and installs it in a hidden subdirectory of your current working directory, named **.terraform**.
- Terraform creates a lock file named **.terraform.lock.hcl** which specifies the exact provider versions used, so that you can control when you want to update the providers used for your project.

CREATE INFRASTRUCTURE

Apply the configuration now with the **terraform apply** command.

INSPECT STATE

- When you applied your configuration, Terraform wrote data into a file called ***terraform.tfstate***.
- Terraform stores the IDs and properties of the resources it manages in this file, so that it can update or destroy those resources going forward.
- The **Terraform state file** is the only way Terraform can track which resources it manages.

INSPECT STATE

In production, storing state remotely with HCP Terraform or Terraform Enterprise is recommended. Terraform also supports several other remote backends you can use to store and manage your state. Inspect the current state using ***terraform show***.

When Terraform created this EC2 instance, it also gathered the resource's metadata from the AWS provider and wrote the metadata to the state file.

<https://developer.hashicorp.com/terraform/tutorials/cloud/cloud-migrate>

<https://developer.hashicorp.com/terraform/language/backend>

APPLY CHANGES

After changing the configuration, run `terraform apply` again to see how Terraform will apply this change to the existing resources.

DESTROY INFRASTRUCTURE

The ***terraform destroy*** command terminates resources managed by your Terraform project.

This command is the inverse of *terraform apply* in that it terminates all the resources specified in your Terraform state.

STORE REMOTE STATE

Now you have built, changed, and destroyed infrastructure from your local machine.

This is great for testing and development, but in production environments you should keep your state secure and encrypted, where your teammates can access it to collaborate on infrastructure.

The best way to do this is by **running Terraform in a remote environment** with shared access to state.

HCP Terraform allows teams to easily version, audit, and collaborate on infrastructure changes.

It also securely stores variables, including API tokens and access keys, and provides a safe, stable environment for long-running Terraform processes.

TERRAFORM MODULES

- A Terraform module is a set of Terraform configuration files in a single directory.
- Even a simple configuration consisting of a single directory with one or more .tf files is a module.
- When you run Terraform commands directly from such a directory, it is considered the **root module**.

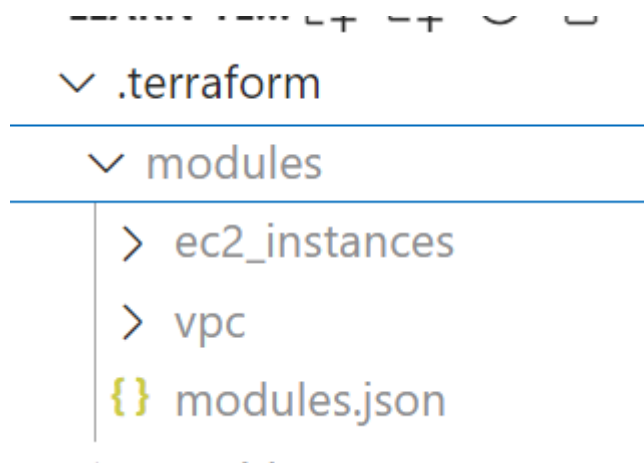
YOU MAY HAVE A SIMPLE SET OF TERRAFORM CONFIGURATION FILES SUCH AS:

.

- |—— LICENSE
- |—— README.md
- |—— main.tf
- |—— variables.tf
- |—— outputs.tf

BUILD AND USE A LOCAL MODULE

<https://developer.hashicorp.com/terraform/tutorials/modules/module-create>







QNA

LETS CONNECT



Sunil Kumar

Corporate Trainer- Cloud
Technologies | AI practitioner | ...



<https://www.linkedin.com/in/cloudwithsunil/>

REFERENCES

https://developer.hashicorp.com/terraform/tutorials/aws-get-started?product_intent=terraform

<https://developer.hashicorp.com/terraform/tutorials/modules/module-use?variants=terraform-workflow%3Acommunity>